

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”

Курс «Парадигмы и конструкции языков программирования»

Отчет по рубежному контролю №2

Вариант А 27

Выполнил:
студент группы ИУ5 - 32Б:
Акопов А.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.
Подпись и дата:

Москва, 2025 г.

Задание:

Рубежный контроль представляет собой разработку тестов на языке Python.

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Код программы:

Файл RK2.py

```
# Рефакторинг файла RK1.py
from operator import itemgetter

class Teacher:
    """Преподаватель"""
    def __init__(self, id, surname, salary, course_id):
        self.id = id
        self.surname = surname
        self.salary = salary
        self.course_id = course_id

class Course:
    """Учебный курс"""
    def __init__(self, id, name):
        self.id = id
        self.name = name

class TeacherCourse:
    """Преподаватели курсов"""
    def __init__(self, teacher_id, course_id):
        self.teacher_id = teacher_id
        self.course_id = course_id

def get_one_to_many(teachers, courses):
    """Соединение данных один-ко-многим"""
    return [(t.surname, t.salary, c.name)
            for c in courses
            for t in teachers
            if t.course_id == c.id]

def get_many_to_many(teachers, courses, teachers_courses):
```

```

"""Соединение данных многие-ко-многим"""
many_to_many_temp = [(c.name, tc.course_id, tc.teacher_id)
                      for c in courses
                      for tc in teachers_courses
                      if c.id == tc.course_id]

return [(t.surname, t.salary, course_name)
        for course_name, course_id, teacher_id in many_to_many_temp
        for t in teachers if t.id == teacher_id]

def task_a1(one_to_many):
    """Задание A1: отсортировать по названию курса"""
    return sorted(one_to_many, key=itemgetter(2))

def task_a2(one_to_many, courses):
    """Задание A2: суммарная зарплата преподавателей по курсам"""
    res_2_unsorted = []
    for c in courses:
        course_teachers = list(filter(lambda i: i[2] == c.name, one_to_many))
        if course_teachers:
            total_salary = sum([salary for _, salary, _ in course_teachers])
            res_2_unsorted.append((c.name, total_salary))

    return sorted(res_2_unsorted, key=itemgetter(1), reverse=True)

def task_a3(many_to_many, courses):
    """Задание A3: курсы со словом 'данных' и их преподаватели"""
    res_3 = {}
    for c in courses:
        if 'данных' in c.name:
            course_teachers = list(filter(lambda i: i[2] == c.name, many_to_many))
            teacher_surnames = [surname for surname, _, _ in course_teachers]
            res_3[c.name] = teacher_surnames
    return res_3

def main():
    """Основная функция с тестовыми данными"""
    # Тестовые данные
    courses = [
        Course(1, 'Математический анализ'),
        Course(2, 'Программирование на Python'),
        Course(3, 'Базы данных'),
        Course(4, 'Алгоритмы и структуры данных'),
        Course(5, 'Физика'),
    ]

```

```

teachers = [
    Teacher(1, 'Иванов', 50000, 1),
    Teacher(2, 'Петров', 60000, 2),
    Teacher(3, 'Сидоров', 55000, 3),
    Teacher(4, 'Кузнецов', 65000, 4),
    Teacher(5, 'Попов', 52000, 5),
    Teacher(6, 'Смирнов', 58000, 1),
    Teacher(7, 'Федоров', 62000, 2),
]

teachers_courses = [
    TeacherCourse(1, 1),
    TeacherCourse(1, 3),
    TeacherCourse(2, 2),
    TeacherCourse(3, 3),
    TeacherCourse(4, 4),
    TeacherCourse(5, 5),
    TeacherCourse(6, 1),
    TeacherCourse(7, 2),
    TeacherCourse(2, 4),
]

# Получаем соединения данных
one_to_many = get_one_to_many(teachers, courses)
many_to_many = get_many_to_many(teachers, courses, teachers_courses)

# Выполняем задания
print('Задание A1')
res_1 = task_a1(one_to_many)
for surname, salary, course_name in res_1:
    print(f'Курс: {course_name}, Преподаватель: {surname}, Зарплата: {salary}')

print('\nЗадание A2')
res_2 = task_a2(one_to_many, courses)
for course_name, total_salary in res_2:
    print(f'Курс: {course_name}, Суммарная зарплата: {total_salary}')

print('\nЗадание A3')
res_3 = task_a3(many_to_many, courses)
for course_name, teachers_list in res_3.items():
    teachers_str = ', '.join(teachers_list) if teachers_list else 'нет преподавателей'
    print(f'Курс: {course_name}, Преподаватели: {teachers_str}')

if __name__ == "__main__":
    main()

```

Файл test_tdd.py

```
# Рефакторинг файла RK1.py
import unittest
from RK2 import Teacher, Course, TeacherCourse, get_one_to_many, get_many_to_many,
task_a1, task_a2, task_a3

class TestRK1(unittest.TestCase):

    def setUp(self):
        """Инициализация тестовых данных перед каждым тестом"""
        self.courses = [
            Course(1, 'Математический анализ'),
            Course(2, 'Программирование на Python'),
            Course(3, 'Базы данных'),
            Course(4, 'Алгоритмы и структуры данных'),
            Course(5, 'Физика'),
        ]

        self.teachers = [
            Teacher(1, 'Иванов', 50000, 1),
            Teacher(2, 'Петров', 60000, 2),
            Teacher(3, 'Сидоров', 55000, 3),
            Teacher(4, 'Кузнецов', 65000, 4),
            Teacher(5, 'Попов', 52000, 5),
            Teacher(6, 'Смирнов', 58000, 1),
            Teacher(7, 'Федоров', 62000, 2),
        ]

        self.teachers_courses = [
            TeacherCourse(1, 1),
            TeacherCourse(1, 3),
            TeacherCourse(2, 2),
            TeacherCourse(3, 3),
            TeacherCourse(4, 4),
            TeacherCourse(5, 5),
            TeacherCourse(6, 1),
            TeacherCourse(7, 2),
            TeacherCourse(2, 4),
        ]

        self.one_to_many = get_one_to_many(self.teachers, self.courses)
        self.many_to_many = get_many_to_many(self.teachers, self.courses,
self.teachers_courses)

    def test_task_a1(self):
        """
        Тест задания A1:
        Проверяем сортировку связей один-ко-многим по названию курса.
        """
```

```

"""
result = task_a1(self.one_to_many)

# Проверяем, что результат отсортирован по названию курса (по алфавиту)
self.assertEqual(result[0][2], 'Алгоритмы и структуры данных')
self.assertEqual(result[1][2], 'Базы данных')
self.assertEqual(result[2][2], 'Математический анализ')

# Проверяем, что сортировка действительно работает
course_names = [item[2] for item in result]
self.assertEqual(course_names, sorted(course_names))

def test_task_a2(self):
    """
    Тест задания A2:
    Проверяем подсчет суммарной зарплаты преподавателей по курсам
    и сортировку по убыванию суммы.
    """
    result = task_a2(self.one_to_many, self.courses)

    # Проверяем, что результат отсортирован по убыванию суммарной зарплаты
    salaries = [item[1] for item in result]
    self.assertEqual(salaries, sorted(salaries, reverse=True))
    # Создаем словарь для удобства проверки
    result_dict = {course: salary for course, salary in result}
    # Проверяем суммарную зарплату для курса "Программирование на Python"
    self.assertEqual(result_dict.get('Программирование на Python'), 122000)
    # Проверяем суммарную зарплату для курса "Математический анализ"
    self.assertEqual(result_dict.get('Математический анализ'), 108000)
    # Проверяем суммарную зарплату для курса "Алгоритмы и структуры данных"
    self.assertEqual(result_dict.get('Алгоритмы и структуры данных'), 65000)

def test_task_a3(self):
    """
    Тест задания A3:
    Проверяем фильтрацию курсов по ключевому слову 'данных'
    и вывод списка преподавателей для этих курсов.
    """
    result = task_a3(self.many_to_many, self.courses)

    self.assertIn('Базы данных', result)
    self.assertIn('Алгоритмы и структуры данных', result)

    # Курсы без слова 'данных' не должны присутствовать
    self.assertNotIn('Математический анализ', result)
    self.assertNotIn('Программирование на Python', result)
    self.assertNotIn('Физика', result)

    # Проверяем преподавателей для курса 'Базы данных'
    # Из данных: Иванов (id=1) и Сидоров (id=3) ведут Базы данных

```

```
teachers_for_db = result.get('Базы данных', [])
self.assertIn('Иванов', teachers_for_db)
self.assertIn('Сидоров', teachers_for_db)
self.assertEqual(len(teachers_for_db), 2)

# Проверяем преподавателей для курса 'Алгоритмы и структуры данных'
# Из данных: Кузнецов (id=4) ведет Алгоритмы, Петров (id=2) тоже ведет Алгоритмы
teachers_for_algo = result.get('Алгоритмы и структуры данных', [])
self.assertIn('Кузнецов', teachers_for_algo)
self.assertIn('Петров', teachers_for_algo)
self.assertEqual(len(teachers_for_algo), 2)

if __name__ == '__main__':
    unittest.main()
```

Скриншот работы:

```
● (base) an.akopov@MacBook-Air-Akopov RK2 % python -m unittest test_tdd.py
...
-----
Ran 3 tests in 0.000s
```